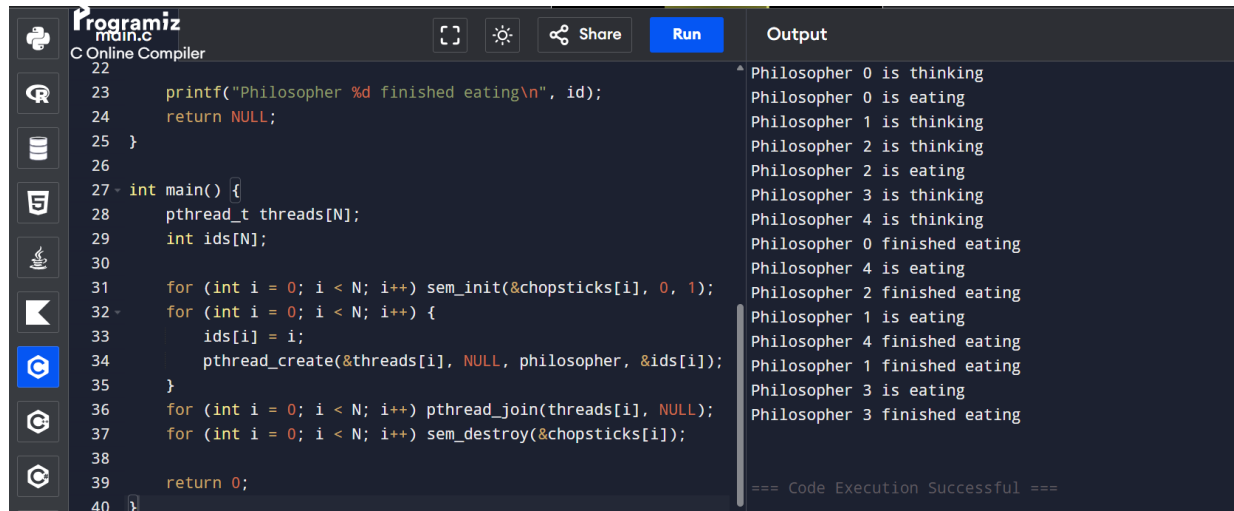


EXPERIMENT 12:

12. Design a C program to simulate the concept of Dining-Philosophers problem.



The screenshot shows a C program in the Programiz online compiler. The code defines a main function that initializes an array of semaphores for 5 chopsticks, creates 5 philosopher threads, and then joins them. The output shows the sequence of actions for 5 philosophers (0 to 4), including thinking, eating, and finishing eating.

```
22
23     printf("Philosopher %d finished eating\n", id);
24     return NULL;
25 }
26
27 int main() {
28     pthread_t threads[N];
29     int ids[N];
30
31     for (int i = 0; i < N; i++) sem_init(&chopsticks[i], 0, 1);
32     for (int i = 0; i < N; i++) {
33         ids[i] = i;
34         pthread_create(&threads[i], NULL, philosopher, &ids[i]);
35     }
36     for (int i = 0; i < N; i++) pthread_join(threads[i], NULL);
37     for (int i = 0; i < N; i++) sem_destroy(&chopsticks[i]);
38
39     return 0;
40 }
```

Output:

```
Philosopher 0 is thinking
Philosopher 0 is eating
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 2 is eating
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 0 finished eating
Philosopher 4 is eating
Philosopher 2 finished eating
Philosopher 1 is eating
Philosopher 4 finished eating
Philosopher 1 finished eating
Philosopher 3 is eating
Philosopher 3 finished eating
=== Code Execution Successful ===
```

PSEUDO CODE :

1. Initialize an array of semaphores for N chopsticks.
2. Create N philosopher threads.
3. Each philosopher thread loops:
 - a. Think
 - b. Wait on left chopstick `sem_wait(&chopstick[id])`
 - c. Wait on right chopstick `sem_wait(&chopstick[(id+1)%N])`
 - d. Eat
 - e. Release right and left chopstick `sem_post()`

CODE:

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define N 5

sem_t chopsticks[N];

void* philosopher(void* num) {

    int id = *(int*)num;

    printf("Philosopher %d is thinking\n", id);

    sem_wait(&chopsticks[id]);

    sem_wait(&chopsticks[(id + 1) % N]);

    printf("Philosopher %d is eating\n", id);

    sleep(1);

    sem_post(&chopsticks[id]);

    sem_post(&chopsticks[(id + 1) % N]);

    printf("Philosopher %d finished eating\n", id);

    return NULL;

}
```

```
int main() {  
    pthread_t threads[N];  
    int ids[N];  
  
    for (int i = 0; i < N; i++) sem_init(&chopsticks[i], 0, 1);  
    for (int i = 0; i < N; i++) {  
        ids[i] = i;  
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);  
    }  
    for (int i = 0; i < N; i++) pthread_join(threads[i], NULL);  
    for (int i = 0; i < N; i++) sem_destroy(&chopsticks[i]);  
  
    return 0;  
}
```